

Basic Embedded & Architecture

❖ What will happen if two interrupt requests arise at the same time? (bonus if stated through ARM)

The CPU can't service both at once, so the interrupt controller decides which one goes first. This is usually done by priority:

- Each interrupt source is assigned to a priority level.
- The controller compares the pending interrupts and picks the highest-priority one to run first.
- The lower-priority one stays pending and gets serviced right after .
- If both interrupts have the same priority, the controller falls back to a fixed tie-breaker usually the interrupt with the lower vector/IRQ number wins.

On ARM: This is handled by the NVIC (Nested Vectored Interrupt Controller). Each interrupt has a programmable priority (lower number = higher priority, e.g. 0 is highest). When two IRQs are pending together:

- NVIC picks the one with the numerically lowest priority value to execute first.
- If priorities are equal, NVIC breaks the tie using the **IRQ number** (lower number wins).
- Because NVIC is "nested," if a new interrupt comes in with a higher priority than the one currently executing, it will preempt it and the CPU jumps to the new ISR, then resumes the old one afterward.

❖ What are communication protocols in the context of embedded systems? Provide at least Three examples.

A communication protocol is a defined set of rules for how data is formatted, timed, and transferred between a microcontroller and other devices (sensors, memory chips, other MCUs, etc.). It decides things like how many wires are used, who initiates communication, and how bits are framed.

Three common examples:

- **UART (Universal Asynchronous Receiver/Transmitter)** : Simple, 2-wire (TX/RX), no shared clock line timing is agreed on beforehand via a baud rate. Good for point-to-point communication, like MCU-to-PC serial debugging.
- **I2C (Inter-Integrated Circuit)** : 2-wire (SDA/SCL), supports multiple devices on the same bus using addresses, master-slave style. Common for connecting sensors (like the BNO055 above) since it needs few pins.
- **SPI (Serial Peripheral Interface)** : 4-wire (MOSI, MISO, SCK, CS), full-duplex and much faster than I2C, but needs a separate chip-select line per device. Common for high-speed peripherals like SD cards, displays, or ADCs.

❖ What are the differences between the following families and their use cases: STM32F4, Raspberry Pi Zero, ATmega.

STM32F4 : A 32-bit ARM Cortex-M4 microcontroller, no OS by default (bare-metal or a small RTOS), decent clock speed (up to ~180 MHz), rich set of built-in peripherals (multiple I2C/SPI/UART, CAN, ADC/DAC, timers).

Used in: Real-time and deterministic. Real-time control tasks where timing matters reading sensors fast and reliably, running control loops, driving a CAN bus. This is the typical "real embedded controller" choice for something like a BSPD board or a sensor acquisition unit.

Raspberry Pi Zero: A tiny single-board computer, runs a full Linux OS, has a real CPU (ARM11) with RAM, storage, networking. Not real-time the OS can introduce unpredictable delays.

Use in: Tasks that benefit from having an OS: data logging to files, running a camera/vision pipeline, networking (Wi-Fi dashboards, streaming telemetry), or anything needing higher-level software (Python scripts, web servers). Not ideal if you need guaranteed microsecond-level timing.

ATmega :A simple 8-bit microcontroller, lower clock speed (~16 MHz), limited RAM/flash, fewer built-in peripherals compared to STM32F4. Great for :simple, low-cost, low-power dedicated tasks reading one or two sensors, blinking status LEDs, small standalone modules. Also very common for beginners/prototyping because of the huge Arduino ecosystem, but it runs out of steam quickly for anything computation- or peripheral-heavy.